

PHÂN TÍCH THUTOÁN ADMM VÀ ÁNH XPHẦN CỨNG FPGA

Chi tiết triển khai Reference Model Float64 (admm_golden_float64.py)
cho bài toán Tối ưu hóa danh mục đầu tư thời gian thực

FPGA Hardware Acceleration Team

Ngày 24 tháng 8 năm 2026

1 Tổng quan bài toán tối ưu hóa danh mục đầu tư

Bài toán tái cân bằng danh mục đầu tư theo thời gian thực có xét đến chi phí giao dịch ℓ_1 và ràng buộc tỷ trọng (Box-Constrained Simplex) được mô hình hóa dưới dạng bài toán quy hoạch toàn phương lồi:

$$\min_{w \in \mathbb{R}^N} \frac{1}{2} w^T \Sigma w - \mu^T w + \lambda \|w - w_{\text{prev}}\|_1 + \mathbb{I}_{\mathcal{C}}(w) \quad (1)$$

Trong đó:

- $\Sigma \in \mathbb{S}_{++}^N$: Ma trận hiệp phương sai lợi suất tài sản (đối xứng xác định dương).
- $\mu \in \mathbb{R}^N$: Vector kỳ vọng lợi suất của N tài sản.
- $w_{\text{prev}} \in \mathbb{R}^N$: Vector tỷ trọng danh mục hiện tại (tại thời điểm $t - 1$).
- $\lambda > 0$: Hệ số phạt chi phí giao dịch ℓ_1 .
- $\mathcal{C} = \left\{ w \in \mathbb{R}^N \mid \sum_{i=1}^N w_i = 1, 0 \leq w_i \leq w_{\text{max}} \right\}$: Tập ràng buộc simplex có chặn (không bán không và giới hạn trần tỷ trọng w_{max}).
- $\mathbb{I}_{\mathcal{C}}(w)$: Hàm chỉ thị ($\mathbb{I}_{\mathcal{C}}(w) = 0$ nếu $w \in \mathcal{C}$, và $+\infty$ nếu ngược lại).

2 Chu trình tính toán ADMM (5 bước hoàn chỉnh)

Kỹ thuật tách biến (Variable Splitting) đưa biến phụ $z \in \mathbb{R}^N$ với ràng buộc $w - z = 0$. Hàm Lagrangian tăng cường dạng thu phóng (Scaled Augmented Lagrangian) với tham số $\rho > 0$ và biến đối ngẫu $u = \frac{1}{\rho} y$ có dạng:

$$\mathcal{L}_{\rho}(w, z, u) = \frac{1}{2} w^T \Sigma w - \mu^T w + \lambda \|z - w_{\text{prev}}\|_1 + \mathbb{I}_{\mathcal{C}}(z) + \frac{\rho}{2} \|w - z + u\|_2^2 - \frac{\rho}{2} \|u\|_2^2 \quad (2)$$

Mỗi vòng lặp k của ADMM được chia thành các bước tính toán đóng (closed-form updates):

1. Tiền xử lý (Khởi tạo / Offline):

- Thiết lập ma trận hệ số: $A = \Sigma + \rho I_N$ (đối xứng xác định dương).
- Phân rã Cholesky-Banachiewicz: $A = L_A L_A^T$ (chỉ tính 1 lần duy nhất).

2. Vòng lặp ADMM (Mỗi iteration $k = 0, 1, \dots, \text{max_iter} - 1$):

1. **Bước 1 (w -update):** Giải hệ phương trình $Aw = \mu + \rho(z^k - u^k)$ qua 2 lần thế:

$$L_A y = \mu + \rho(z^k - u^k) \longrightarrow \text{tìm } y \text{ (Thế xuôi - Forward Solve)} \quad (3)$$

$$L_A^T w^{k+1} = y \longrightarrow \text{tìm } w^{k+1} \text{ (Thế ngược - Backward Solve)} \quad (4)$$

2. **Bước 2 (z -update (a) - Soft-Thresholding):** Tính toán co mềm theo chi phí ℓ_1 :

$$v = w^{k+1} + u^k, \quad \Delta = v - w_{\text{prev}} \quad (5)$$

$$\tilde{z}_i = w_{\text{prev},i} + \text{sign}(\Delta_i) \cdot \max\left(|\Delta_i| - \frac{\lambda}{\rho}, 0\right) \quad (6)$$

3. **Bước 3 (z -update (b) - Simplex Projection Π_C):** Chiếu vector $\tilde{z}$ lên tập C :

$$z_i^{k+1} = \text{clip}(\tilde{z}_i - \nu^*, 0, w_{\text{max}}) \quad \text{sao cho} \quad \sum_{i=1}^N \text{clip}(\tilde{z}_i - \nu^*, 0, w_{\text{max}}) = 1 \quad (7)$$

4. **Bước 4 (u -update - Dual Accumulation):** Tích lũy sai số đối ngẫu:

$$u^{k+1} = u^k + (w^{k+1} - z^{k+1}) \quad (8)$$

5. **Bước 5 (Stopping Check - Điều kiện dừng KKT):**

$$r_{\text{pri}} = \|w^{k+1} - z^{k+1}\|_{\infty} \leq \epsilon_{\text{pri}} \quad \wedge \quad s_{\text{dual}} = \rho \|z^{k+1} - z^k\|_{\infty} \leq \epsilon_{\text{dual}} \quad (9)$$

3 Bảng đối chiếu bước tính & Ảnh xạ phần cứng FPGA

Bước ADMM	Biểu thức Toán học	Khối Phần cứng (HW Block)	Độ phức tạp
w -update	Giải $(L_A L_A^T)w = \mu + \rho(z - u)$	1D Folded Systolic Array (Dual DSP MAC)	$\mathcal{O}(N^2/P)$
z -update (a)	$\tilde{z} = \mathcal{S}_{\lambda/\rho}(w + u - w_{\text{prev}}) + w_{\text{prev}}$	SIMD Soft-Thresholding Core	$\mathcal{O}(N/P)$
z -update (b)	$z = \Pi_C(\tilde{z})$	Bitonic Sorting + Prefix Water-Filling	$\mathcal{O}(\log_2^2 N)$
u -update	$u \leftarrow u + (w - z)$	Dual Accumulator DSP48E2	$\mathcal{O}(N/P)$
Stopping Check	$\ w - z\ _{\infty} \leq \epsilon_{\text{pri}} \quad \wedge \quad \rho \ z - z_{\text{old}}\ _{\infty} \leq \epsilon_{\text{dual}}$	Pipelined Tree Comparator	$\mathcal{O}(\log_2 N)$

Bảng 1: Đối chiếu giữa các bước thuật toán và kiến trúc phần cứng trên Xilinx KV260.

4 Chi tiết từng hàm trong admm_golden_float64.py

4.1 cholesky_factorize(A: np.ndarray) -> np.ndarray

- **Mục đích:** Phân rã ma trận đối xứng xác định dương $A = \Sigma + \rho I \in \mathbb{R}^{N \times N}$ thành ma trận tam giác dưới L thỏa mãn $A = LL^T$ bằng thuật toán Cholesky-Banachiewicz.

- **Công thức tính toán:**

$$L_{i,i} = \sqrt{A_{i,i} - \sum_{k=0}^{i-1} L_{i,k}^2}, \quad \forall i = 0, \dots, N-1 \quad (10)$$

$$L_{i,j} = \frac{1}{L_{j,j}} \left(A_{i,j} - \sum_{k=0}^{j-1} L_{i,k} L_{j,k} \right), \quad \forall i > j \quad (11)$$

- **Nguyên lý hoạt động:** Duyệt từng hàng i và cột $j \leq i$. Tích vô hướng $\sum L_{i,k} L_{j,k}$ được tính lũy tiến từ các phần tử đã biết. Nếu trên đường chéo ($i = j$), kiểm tra điều kiện dương $A_{i,i} - \text{sum} > 0$ rồi khai căn. Nếu ngoài đường chéo ($i > j$), chia cho phần tử đường chéo $L_{j,j}$.
- **Ý nghĩa hiệu năng:** Chỉ cần thực thi một lần duy nhất lúc khởi tạo hệ thống vì ma trận A cố định qua các vòng lặp ADMM.

4.2 forward_solve(L: np.ndarray, b: np.ndarray) -> np.ndarray

- **Mục đích:** Giải hệ ma trận tam giác dưới $Ly = b$ tìm vector trung gian $y \in \mathbb{R}^N$ bằng phép thế xuôi (Forward Substitution).
- **Công thức tính toán:**

$$y_i = \frac{1}{L_{i,i}} \left(b_i - \sum_{j=0}^{i-1} L_{i,j} y_j \right), \quad i = 0, 1, \dots, N-1 \quad (12)$$

- **Nguyên lý hoạt động:** Duyệt i từ $0 \rightarrow N-1$. Tại mỗi bước, trừ đi tích lũy các biến y_j ($j < i$) đã biết từ trước, sau đó chia cho hệ số $L_{i,i}$.
- **Ánh xạ phần cứng:** Mảng Systolic Array 1D với các phần tử xử lý song song (PE) sử dụng bộ nhân-cộng DSP48E2.

4.3 backward_solve(L: np.ndarray, y: np.ndarray) -> np.ndarray

- **Mục đích:** Giải hệ ma trận tam giác trên $L^T w = y$ tìm vector nghiệm $w \in \mathbb{R}^N$ bằng phép thế ngược (Backward Substitution).
- **Công thức tính toán:**

$$w_i = \frac{1}{L_{i,i}} \left(y_i - \sum_{j=i+1}^{N-1} L_{j,i} w_j \right), \quad i = N-1, N-2, \dots, 0 \quad (13)$$

- **Nguyên lý hoạt động:** Duyệt ngược i từ $N-1 \rightarrow 0$. Trừ đi tích lũy các biến w_j ($j > i$) đã biết ở các hàng phía dưới, sau đó chia cho $L_{i,i}$.

4.4 linear_system_solve(L: np.ndarray, b: np.ndarray) -> np.ndarray

- **Mục đích:** Thực hiện trọn gói giải hệ phương trình tuyến tính đối xứng xác định dương $Aw = b$ qua 2 bước thể liên tiếp:

$$w = \text{backward_solve}(L, \text{forward_solve}(L, b)) \quad (14)$$

4.5 soft_threshold(v: np.ndarray, w_prev: np.ndarray, threshold: float) -> np.ndarray

- **Mục đích:** Tính toán toán tử tiệm cận (Proximal Operator) giải quyết hàm phạt chi phí giao dịch phi trơn $\lambda \|z - w_{\text{prev}}\|_1$.

- **Công thức tính toán:** Với $\tau = \frac{\lambda}{\rho}$ và $\Delta = v - w_{\text{prev}}$:

$$\tilde{z}_i = w_{\text{prev},i} + \text{sign}(\Delta_i) \cdot \max(|\Delta_i| - \tau, 0) \quad (15)$$

- **Ý nghĩa kinh tế/tài chính:**

- Nếu tín hiệu giao dịch nhỏ ($|\Delta_i| \leq \tau$): Chi phí giao dịch lớn hơn lợi ích dự kiến $\Rightarrow$ Giữ nguyên tỷ trọng $\tilde{z}_i = w_{\text{prev},i}$.
- Nếu tín hiệu giao dịch đủ lớn ($|\Delta_i| > \tau$): Có lợi nhuận ròng sau chi phí $\Rightarrow$ Điều chỉnh tỷ trọng theo hướng tín hiệu một lượng bằng τ .

- **Ánh xạ phần cứng:** Khối SIMD Soft-Thresholding Core tính toán đồng thời song song trên toàn bộ N tài sản chỉ trong 1 chu kỳ clock.

4.6 exact_simplex_projection(z_tilde, w_max, total_budget=1.0) -> Tuple[np.ndarray, float]

- **Mục đích:** Thực hiện phép chiếu Euclid trực giao chính xác từ vector $\tilde{z}$ lên tập đa diện lồi ràng buộc simplex có chặn $\mathcal{C}$.
- **Nguyên lý bài toán:** Tìm nhân tử Lagrange đối ngẫu ν^* thỏa mãn:

$$z_i = \text{clip}(\tilde{z}_i - \nu^*, 0, w_{\text{max}}) = \max(0, \min(\tilde{z}_i - \nu^*, w_{\text{max}})) \quad (16)$$

sao cho hàm tổng ngân sách đơn điệu giảm theo ν :

$$\Phi(\nu) = \sum_{i=1}^N \text{clip}(\tilde{z}_i - \nu, 0, w_{\text{max}}) = 1.0 \quad (17)$$

- **Thuật toán Breakpoint / Water-Filling:**

1. Xác định tập điểm gãy $\mathcal{B} = \{\tilde{z}_i\} \cup \{\tilde{z}_i - w_{\text{max}}\}$.
2. Sắp xếp giảm dần tập $\mathcal{B}$ để chia trực số thực thành các khoảng tuyến tính.
3. Tìm đoạn $[\nu_{\text{low}}, \nu_{\text{high}}]$ chứa giá trị $\Phi(\nu) = 1.0$.
4. Tính nghiệm giải tích chính xác của ν^* trong đoạn tích cực (active set):

$$\nu^* = \frac{\sum_{i \in \text{active}} \tilde{z}_i + \sum_{i \in \text{upper}} w_{\text{max}} - 1}{|\text{active}|} \quad (18)$$

5. Nếu nằm ngoài miền, kích hoạt thuật toán chia đôi (Bisection) dự phòng để đảm bảo độ chính xác tuyệt đối.

- **Ánh xạ phần cứng:** Mạng trộn sắp xếp Bitonic Sorting Network $\mathcal{O}(\log_2^2 N)$ kết hợp khối cộng dồn tiền tố (Prefix Water-Filling) tạo pipeline hoàn toàn không có chu trình chờ (no stalling).

4.7 admm_portfolio_solve(...)

- **Mục đích:** Hàm điều phối chính (Top-level Solver) kết nối toàn bộ chu trình tính toán:

1. Khởi tạo $w = w_{\text{prev}}, z = w_{\text{prev}}, u = \mathbf{0}$.
2. Tính phân rã Cholesky L_A từ $A = \Sigma + \rho I$.
3. Lặp qua từng iteration k :
 - $w \leftarrow \text{linear_system_solve}(L_A, \mu + \rho(z - u))$
 - $\tilde{z} \leftarrow \text{soft_threshold}(w + u, w_{\text{prev}}, \lambda/\rho)$
 - $z \leftarrow \text{exact_simplex_projection}(\tilde{z}, w_{\text{max}})$
 - $u \leftarrow u + (w - z)$
 - Đánh giá sai số hội tụ r_{pri} và s_{dual} .
4. Trả về tỷ trọng tối ưu $w^* = z$ cùng lịch sử hội tụ.

5 Kết luận

Kiến trúc thuật toán ADMM trên mô hình chuẩn `admm_golden_float64.py` được thiết kế theo cấu trúc mô-đun hóa chặt chẽ, tối ưu hóa triệt để cho việc ánh xạ trực tiếp sang phần cứng SystemVerilog trên FPGA AMD Xilinx Kria KV260:

- Tách biệt hoàn toàn pha tính toán nặng Cholesky ra ngoài vòng lặp.
- Giảm độ phức tạp mỗi vòng lặp xuống mức tuyến tính và logarit nhờ mảng Systolic Array và mạng sắp xếp Bitonic.
- Đảm bảo nghiệm tối ưu luôn khả thi nghiêm ngặt (Strictly Feasible) trên tập lồi Simplex $\mathcal{C}$.